PostgreSQL

SQL Project

Pizza Hut



Qurat Ul Ain Nadeem

Project Overview

01 Introduction

02 Step by Step Approach

03 Dataset, Table Design & ERD

04 Problem Solving & Learnings



Introduction

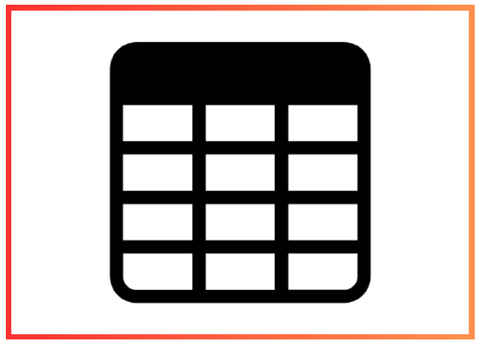
Pizza Hut is presented as a fictional company used as the business scenario. It's a pizza restaurant business that sells pizzas in several categories and sizes, records every order with date, time, pizzas and quantity, and wants decisions backed by sales data. I used no real company facts.

What the business wants

- Best Seller
- Peak Times
- Revenue
- Growth

Step by Step Approach

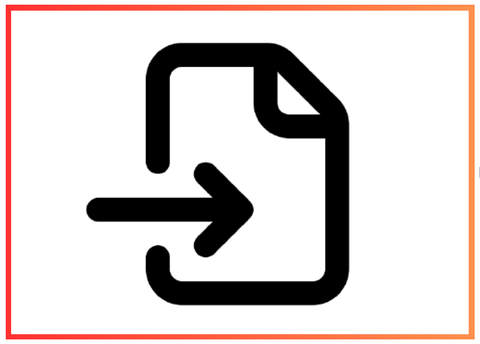
Step 1



DESIGN

Created Four tables with primary and foreign keys and defined their types

Step 2



IMPORT

Loaded CSV files (parent tables before child tables)

Load order:
pizza_types
pizzas
orders
order_details

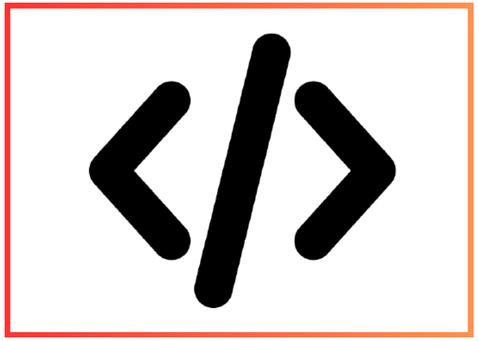
Step 3



VERIFICATION

Verified Loaded Data Files by checking load count

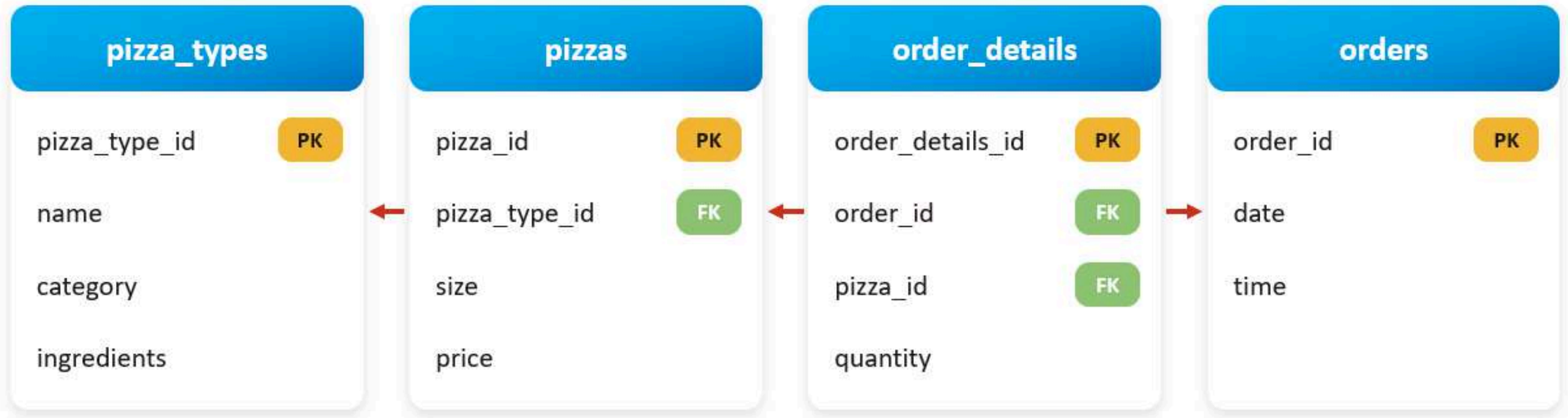
Step 4



SQL QUERIES

Wrote SQL queries from basic to advanced to solve problems

Dataset and table design



PK = Primary Key ， FK = Foreign Key (arrow points to the parent table)

ERD DIAGRAM


public orders
date
order_id
time
public pizzas
pizza_id
pizza_type_id
price
size
public order_details
order_details_id
order_id
pizza_id
quantity
public pizza_types
category
ingredients
name
pizza_type_id
1
*

Problem Statement

TOTAL ORDERS AND TOTAL REVENUE

How many orders were placed?

```
SELECT COUNT(order_id) AS total_orders
FROM orders;
```

What is the total revenue from pizza sales?

```
SELECT ROUND(SUM(od.quantity * p.price), 2)
AS revenue
FROM order_details od
JOIN pizzas p ON p.pizza_id = od.pizza_id;
```

COUNT JOIN SUM ROUND

Problem Statement

HIGHEST PRICE AND MOST COMMON SIZE

Which pizza has the highest price?

```
SELECT pt.name, p.price
FROM pizza_types pt
JOIN pizzas p
ON p.pizza_type_id = pt.pizza_type_id
ORDER BY p.price DESC
LIMIT 1;
```

Which pizza size is ordered most often?

```
SELECT p.size,
COUNT(od.order_details_id) AS order_count
FROM order_details od
JOIN pizzas p ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY 2 DESC;
```

JOIN | ORDER BY | LIMIT | JOIN | COUNT | GROUP BY

Problem Statement

TOP FIVE MOST ORDERED PIZZA TYPES

Which pizza has the highest price?

```
SELECT pt.name, SUM(od.quantity) AS qty
FROM pizza_types pt
JOIN pizzas p
ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY qty DESC
LIMIT 5;
```

WHAT IT DOES

- Joins pizza_types, pizzas and order_details
- Adds up quantity for each pizza name
- Sorts high to low and keeps 5 rows

LIMIT JOIN SUM GROUP BY

Problem Statement

CATEGORY QUANTITY AND DISTRIBUTION

Total quantity ordered in each pizza category

```
SELECT pt.category,
    SUM(od.quantity) AS pizza_count
FROM pizza_types pt
JOIN pizzas p
ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
ON p.pizza_id = od.pizza_id
GROUP BY pt.category
ORDER BY pizza_count DESC;
```

Category-wise distribution of pizzas

```
SELECT category, COUNT(name)
FROM pizza_types
GROUP BY category;
```

JOIN SUM GROUP BY COUNT GROUP BY

Problem Statement

ORDERS BY HOUR AND DAILY AVERAGE

Distribution of orders by hour of the day

```
SELECT EXTRACT(HOUR FROM time) AS
order_hour,
    COUNT(order_id) AS orders_hour
FROM orders
GROUP BY EXTRACT(HOUR FROM time)
ORDER BY order_hour DESC;
```

Average number of pizzas ordered per day

```
SELECT ROUND(AVG(t_qty), 2) AS average_qty
FROM (
SELECT o.date, SUM(od.quantity) AS t_qty
FROM orders o
JOIN order_details od
ON o.order_id = od.order_id
GROUP BY o.date
) AS order_qty;
```

EXTRACT | COUNT | GROUP BY | Subquery | AVG | ROUND

Problem Statement

TOP THREE PIZZA TYPES BY REVENUE

```
SELECT pt.name,
    SUM(od.quantity * p.price) AS revenue
FROM pizza_types pt
JOIN pizzas p
ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 3;
```

WHAT IT DOES

- Revenue = quantity x price
- Grouped by pizza name
- Highest 3 kept with LIMIT

LIMIT JOIN SUM GROUP BY

Problem Statement

REVENUE CONTRIBUTION SHARE

```
SELECT pt.category,
ROUND(SUM(od.quantity * p.price) /
(SELECT SUM(od.quantity * p.price)
FROM order_details od
JOIN pizzas p ON p.pizza_id = od.pizza_id)
* 100, 2) AS percentage
FROM pizza_types pt
JOIN pizzas p
ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
ON p.pizza_id = od.pizza_id
GROUP BY pt.category;
```

WHAT IT DOES

- Subquery returns total revenue
- revenue / total x 100 gives the share
- ROUND keeps 2 decimals

Subquery ROUND GROUP BY

Problem Statement

CUMULATIVE REVENUE OVER TIME

```
SELECT a.date,
    SUM(revenue) OVER (ORDER BY a.date) AS
cum
FROM (
SELECT o.date,
      SUM(od.quantity * p.price) AS revenue
FROM orders o
JOIN order_details od
ON o.order_id = od.order_id
JOIN pizzas p
ON p.pizza_id = od.pizza_id
GROUP BY o.date
) AS a;
```

WHAT IT DOES

- Inner query: revenue for each day
- SUM() OVER (ORDER BY date) builds a running total
- Outer query reads the alias a

JOIN | Window function | Subquery

Problem Statement

TOP THREE PIZZAS IN EACH CATEGORY

```
SELECT category, name, rev, rankings
FROM (
SELECT category, name, rev,
RANK() OVER (PARTITION BY category
            ORDER BY rev DESC) AS rankings
FROM (
SELECT pt.category, pt.name,
       SUM(od.quantity * p.price) AS rev
FROM pizza_types pt
JOIN pizzas p
   ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
   ON od.pizza_id = p.pizza_id
GROUP BY pt.category, pt.name
) AS r
) AS b
WHERE rankings <= 3;
```

WHAT IT DOES

- Revenue for each pizza and category
- RANK( ) restarts inside each category
- WHERE keeps ranks 1 to 3

RANK | Nested subquery | PARTITION BY

WHAT I LEARNED

- Multi-table JOINs
- Clean data import
- PostgreSQL syntax
- Aggregation
- Window functions
- Subqueries

Move forward.
Good things are
up ahead

Thank You

Qurat Ul Ain Nadeem